

CHAPTER 5

SIMPLIFICATION OF BOOLEAN FUNCTIONS

5.1 Introduction

In section 3.4, we introduced the logic diagram of a Boolean function. For implementing a sum of product expression, each product term is implemented by an AND gate. The outputs of the AND gates are then ORed with a multi-input OR gate. Thus, the number of AND gates is equal to the number of product terms in the expression. On the other hand, for implementing a product of sums expression, each sum term is implemented by an OR gate. The outputs of the OR gates are then ANDed with a multi-input AND gate. Thus, the number of OR gates is equal to the number of sum terms in the expression. Moreover, the number of input connections in both the cases is equal to the number of literals in the expression. These facts are shown in Figure 5.1. Note that in the logic diagram of Figure 5.1, it is assumed that all of the input variables are simultaneously available in both their uncomplemented (normal) and complemented form. To show this, NOT gates are used with all the inputs to indicate their complemented output. For this reason, the NOT gates are excluded from determining the cost of the logic circuit. The same convention of showing both the uncomplemented and complemented forms of input variables will be followed throughout the book unless otherwise stated.

The number of AND gates in sum of products implementation and the number of OR gates in product of sums implementation constitute the major cost of the logic circuit implementation. Moreover, an increased number of input connections decreases the reliability of the circuit, because outage of a single input connection will make the circuit malfunctioning. In section 3.4, we also found that algebraic expression of a Boolean function is not unique, that is, it is possible to find more than one algebraic expression that specify the same function. Therefore, it is important to find out the expression with minimum number of terms and with minimum number of literals. Finding such an expression is referred to as *simplification* or *minimization* of Boolean functions.

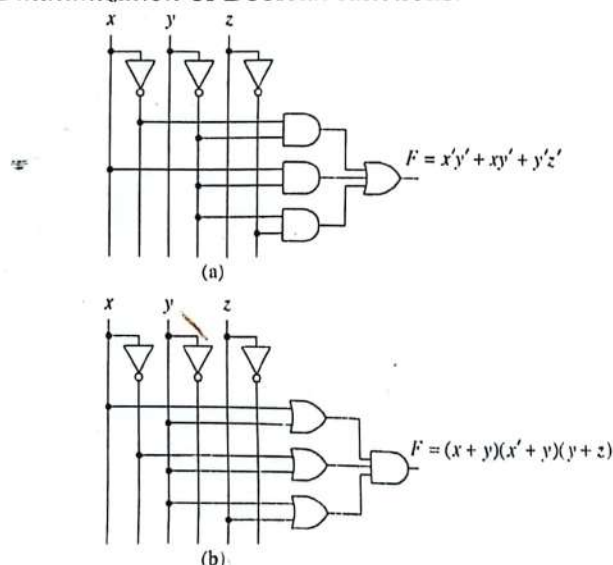


Figure 5.1 Logic circuit implementation of (a) $F = x'y' + xy' + y'z'$ and (b) $F = (x + y)(x' + y)(y + z)$.

In this chapter, we shall discuss four major approaches for simplification of Boolean functions: *algebraic simplification*, *Karnaugh-map method of simplification*, *Quine-McCluskey method of simplification*, and *iterated consensus method of simplification*.

5.2 Algebraic Simplification of Boolean Functions

The Boolean algebra postulates and theorems listed in Table 3.4 can be used to simplify Boolean functions. Unfortunately, it is not always obvious which postulates and theorems should be applied in order to produce the simplest result. Furthermore, there is no easy way to tell whether the simplified expression is in its simplest form or whether it could have been simplified further. Thus, algebraic simplification often becomes a process of trial and error. The quality of algebraic simplification greatly depends on inspiration and experience.

Though, there is no straightforward way of algebraic simplification, the following two steps can be followed:

1. The original expression is put into the sum of products form by using DeMorgan's theorem and distributive law.
2. Once it is in sum of products form, the product terms are checked for common factors, and factoring is performed whenever possible. Sometimes, the expression is expanded by duplicating some product terms to ease factoring. Then postulate and theorems of Boolean algebra are used to eliminate one or more terms.

The above steps are illustrated using the following examples.

Example 1: $F = ABC + AB'(A'C)'$

$$\begin{aligned}
 &= ABC + AB'(A + C) && \text{by theorem 5 (DeMorgan's theorem)} \\
 &= ABC + AB' + AB'C && \text{by postulate 4 (distributive law)} \\
 &= AC(B + B') + AB' && \text{by postulate 4} \\
 &= AC + AB' && \text{by postulate 5}
 \end{aligned}$$

Example 2: $F = ABC + ABC' + AB'C$

$$\begin{aligned}
 &= \underline{ABC} + ABC' + ABC + AB'C && \text{by expanding using theorem 1} \\
 &= AB(C + C') + AC(B + B') && \text{by postulate 4 (distributive law)} \\
 &= AB + AC && \text{by postulate 5}
 \end{aligned}$$

Example 3: $F = A'C(A'BD)' + A'BC'D' + AB'C$

$$\begin{aligned}
 &= A'C(A + B' + D') + A'BC'D' + AB'C && \text{by theorem 5 (DeMorgan's theorem)} \\
 &= A'CA + A'CB' + A'CD' + A'BC'D' + AB'C && \text{by postulate 4 (Distributive law)} \\
 &= A'B'C + A'CD' + A'BC'D' + AB'C && \text{using postulate 5}
 \end{aligned}$$

$$= B'C(A' + A) + A'CD' + A'BC'D' \quad \text{using postulate 4}$$

$$= B'C + A'CD' + A'BC'D' \quad \text{using postulate 5}$$

5.3 Prime Implicants

Karnaugh Map method, Quine-McCluskey method, and iterated consensus method of simplification of Boolean functions will be discussed in the next three sections. Before that concept of prime implicants is worth discussing.

Implication

A Boolean function F_1 implies a Boolean function F_2 (or conversely a Boolean function F_2 is implied by a Boolean function F_1), if $F_2 = 1$ whenever $F_1 = 1$ and F_2 may be either 0 or 1 when $F_1 = 0$. This is written as $F_1 \subseteq F_2$ and read as " F_1 implies F_2 " or " F_1 is included in F_2 " or " F_1 is covered by F_2 " or " F_2 covers F_1 ". F_1 is said to be the implicant of F_2 . As an example, consider the following functions:

$$F_1(A, B, C) = \sum(0, 2, 5, 7)$$

$$F_2(A, B, C) = \sum(0, 5)$$

$$F_3(A, B, C) = \sum(0, 1, 3, 5, 7)$$

It can be noticed that $F_2 \subseteq F_1$, $F_2 \subseteq F_3$, but $F_3 \not\subseteq F_1$, $F_1 \not\subseteq F_3$.

From the definition of implication, the following properties may be noted.

- (1) If $F_1 \subseteq F_2$, then $F_1 + F_2 = F_2$ and $F_1 \cdot F_2 = F_1$.
- (2) If T , T_1 , and T_2 are product terms such that $T = T_1 \cdot T_2$, then $T \subseteq T_2$, since $T + T_2 = T_1 \cdot T_2 + T_2 = T_2$.

Prime Implicants

A product term P is a *prime implicant (PI)* of a Boolean function F if

- (1) $P \subseteq F$, and
- (2) if any literal is deleted from P so as to produce a new product term Q then $Q \not\subseteq F$.

Consider the Boolean function

$$F(A, B, C) = ABC + ABC' + A'BC + AB'C + A'B'C'$$

After simplification it gives

$$F(A, B, C) = AB + BC + AC + A'B'C'$$

Now ABC , ABC' , $A'BC$, and $AB'C$ cannot be prime implicants of the function F , since it is possible to delete a literal from them and still the resulting term is included in F . But AB , BC , AC , and $A'B'C'$ are prime implicants of the function, since it satisfies the condition (1) and (2). If any of the literals is deleted from these terms, the resulting term is not included in F .

A Prime implicant P is said to cover a set of minterms $T_1, T_2, \dots, T_m$ if $T_1 + T_2 + \dots + T_m \subseteq P$. A Boolean function is covered by a set of product terms (usually prime implicants) if the sum of those product terms (prime implicants) equals the function. A minimal cover is obtained by selecting the minimum number of prime implicants of the function.

The term *essential prime implicant* denotes a prime implicant that covers at least one minterm (not a don't care term) that can not be covered by any other prime implicant. The essential prime implicants, therefore, must be included to get a minimum cover of the function. The prime implicants excluding the essential prime implicants are the *non-essential prime implicants*. After inclusion of the essential prime implicants in the cover of the function, we are left with some minterms to be covered by the set of non-essential prime implicants. All of the non-essential prime implicants are not usually needed for this purpose and a subset of the non-essential prime implicants is sufficient for covering the remaining minterms. These prime implicants are known as *secondary-essential prime implicants*.

Minimization of Boolean Functions

The minimization procedure for Boolean functions consists of the following steps:

- (1) The determination of all prime implicants of the function.
- (2) Finding out the essential prime implicants.
- (3) From the non-essential prime implicants, finding out a minimum set of secondary-essential prime implicants so that together with essential prime implicants they cover the function.

5.4 Karnaugh Map Method for Simplification of Boolean Functions

5.4.1 The Karnaugh Map

The *Karnaugh map* (*K-map* in short) is a special pictorial form of a truth table. This map provides a simple straightforward procedure for representation and simplification of Boolean functions. This map method is first proposed by Veitch and then slightly modified by Karnaugh. Perhaps the K-map is the most extensively used tool for simplification of Boolean functions. Although the technique may be used for any number of variables, it is generally used up to six variables beyond which it becomes very cumbersome. Moreover, K-map method of simplification for five and six variables are not so simple to use, they can best be done by Quine-McCluskey method or iterated consensus method. In this section the structure of two-, three-, and four-variable K-maps are presented.

The K-map is a diagram made up of 2^n cells corresponding to 2^n rows of a truth table for n -variable function. The structures of two-, three-, and four-variable K-maps are shown in Figure 5.2. Each cell corresponds to one of the 2^n combinations of n variables and for each row of the truth table there is one specific cell in the K-map. The variables have been designated as A , B , C , and D , and the binary numbers formed by them are taken as AB , ABC , and $ABCD$ for two, three, and four variables respectively. In each map the variables and all possible values of the variables are indicated (the first bit corresponds to the first variable and the second bit corresponds to the second variable) to identify the cells. Gray code sequence has been used for the identification of cells. The reason for using Gray code sequence will be discussed in subsequent discussions. The decimal number corresponding to each cell is written inside the cell in Figure 5.2.

B 0 1			BC 00 01 11 10				CD 00 01 11 10																																																																																						
A	0	<table> <tr><td>0</td><td>1</td></tr> <tr><td>2</td><td>3</td></tr> </table>	0	1	2	3	A	0	<table> <tr><td>0</td><td>1</td><td>3</td><td>2</td></tr> <tr><td>4</td><td>5</td><td>7</td><td>6</td></tr> </table>	0	1	3	2	4	5	7	6	AB	00	<table> <tr><td>0</td><td>1</td><td>3</td><td>2</td></tr> <tr><td>4</td><td>5</td><td>7</td><td>6</td></tr> <tr><td>12</td><td>13</td><td>15</td><td>14</td></tr> <tr><td>8</td><td>9</td><td>11</td><td>10</td></tr> </table>	0	1	3	2	4	5	7	6	12	13	15	14	8	9	11	10	AB	01	<table> <tr><td>0</td><td>1</td><td>3</td><td>2</td></tr> <tr><td>4</td><td>5</td><td>7</td><td>6</td></tr> <tr><td>12</td><td>13</td><td>15</td><td>14</td></tr> <tr><td>8</td><td>9</td><td>11</td><td>10</td></tr> </table>	0	1	3	2	4	5	7	6	12	13	15	14	8	9	11	10	AB	11	<table> <tr><td>0</td><td>1</td><td>3</td><td>2</td></tr> <tr><td>4</td><td>5</td><td>7</td><td>6</td></tr> <tr><td>12</td><td>13</td><td>15</td><td>14</td></tr> <tr><td>8</td><td>9</td><td>11</td><td>10</td></tr> </table>	0	1	3	2	4	5	7	6	12	13	15	14	8	9	11	10	AB	10	<table> <tr><td>0</td><td>1</td><td>3</td><td>2</td></tr> <tr><td>4</td><td>5</td><td>7</td><td>6</td></tr> <tr><td>12</td><td>13</td><td>15</td><td>14</td></tr> <tr><td>8</td><td>9</td><td>11</td><td>10</td></tr> </table>	0	1	3	2	4	5	7	6	12	13	15	14	8	9	11	10
0	1																																																																																												
2	3																																																																																												
0	1	3	2																																																																																										
4	5	7	6																																																																																										
0	1	3	2																																																																																										
4	5	7	6																																																																																										
12	13	15	14																																																																																										
8	9	11	10																																																																																										
0	1	3	2																																																																																										
4	5	7	6																																																																																										
12	13	15	14																																																																																										
8	9	11	10																																																																																										
0	1	3	2																																																																																										
4	5	7	6																																																																																										
12	13	15	14																																																																																										
8	9	11	10																																																																																										
0	1	3	2																																																																																										
4	5	7	6																																																																																										
12	13	15	14																																																																																										
8	9	11	10																																																																																										
1		1		01		11		10																																																																																					

(a) Two-variable K- (b) Three-variable K-map (p) Four-variable K-map

Figure 5.2 The Karnaugh map.

Figure 5.3 shows the minterm/maxterm corresponding to each cell and the term is written inside the cell for clear understanding. Notice that for minterm a variable appears primed if the corresponding marking is 0 and unprimed if 1. Also notice that for maxterm a variable appears primed if the corresponding marking is 1 and unprimed if 0.

The columns and rows of the K-map are marked using the gray code sequence. The characteristic of this sequence is that only one bit changes from 1 to 0 or from 0 to 1 in adjacent cells. *Adjacent cells* are defined to be cells horizontally or vertically next to each other. In addition, the map is considered to lie on a surface with the top and bottom edges, as well as the right and left edges, touching each other to form adjacent cells. For example, in a four-variable K-map, cell 0000 and cell 0010 form adjacent cells, as do cell 0011 and cell 1011.

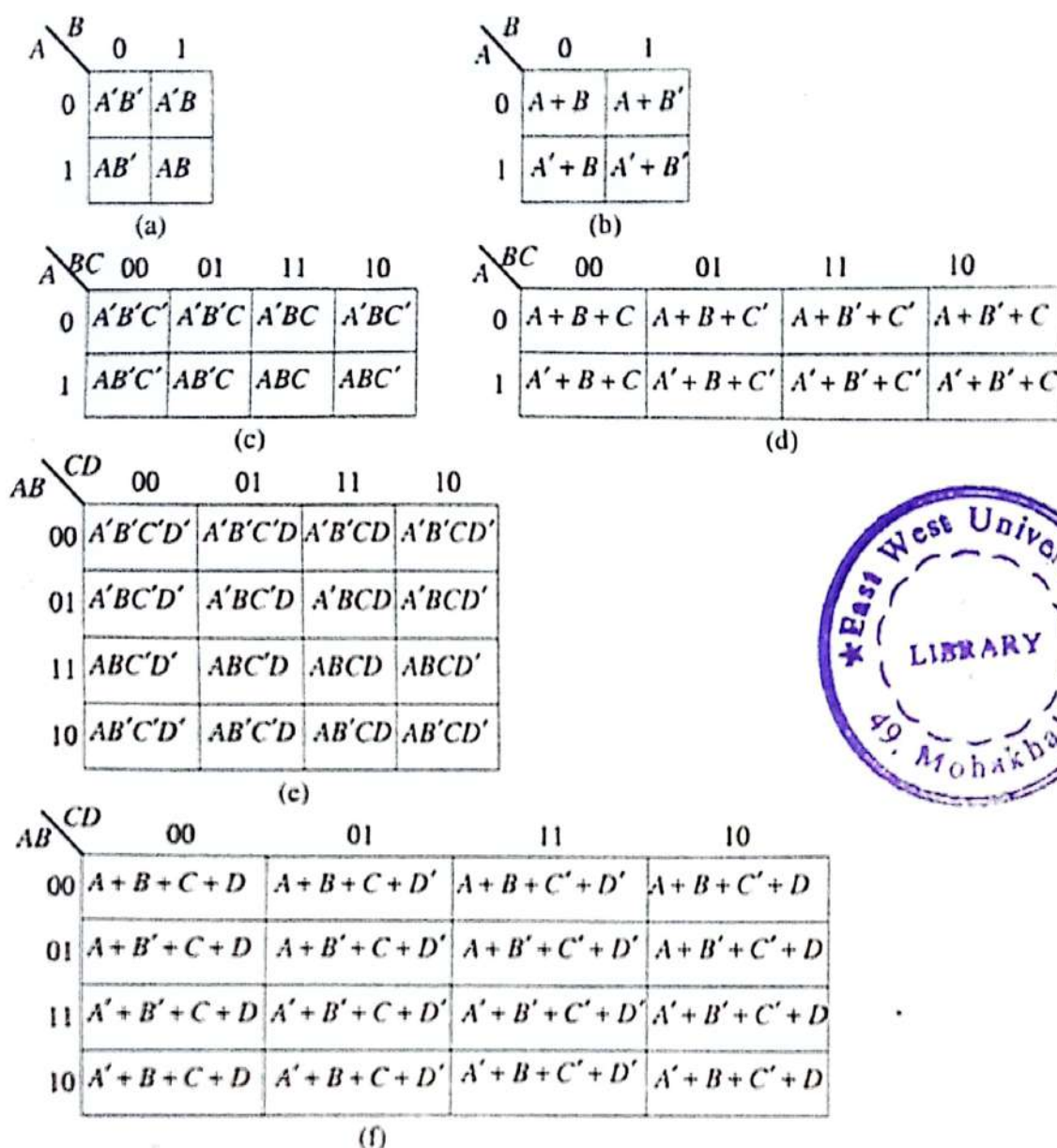


Figure 5.3 Minterm/maxterm corresponding to each cell of K-maps.

5.4.2 Representation of Truth Table on a K-map

For representing a truth table on a K-map, the value of the function (0, 1, or x) for each row of the truth table is entered inside the cell corresponding to that row. Figure 5.4 shows a truth table for a three-variable function and its representation on a three-variable K-map. The cell on the K-map corresponding to a row of the truth table is identified by concatenating the row marking and the column marking of the K-map and then comparing that value with the row of the truth table. For example, in Figure 5.4(b), the second row marking is 1 and the third column marking is 11; their concatenation results in 111. Therefore, the cell common to the second row (1) and the third column (11) corresponds to the row 111 of the truth table. In the example of Figure 5.4, value of the function for the row 111 in the truth table is 1 and a 1 is entered in the identified cell. Alternatively, the cell can be identified by using the equivalent decimal number of the row of the truth table and comparing it

with the cell number as shown in Figure 5.2. For example, the decimal equivalent of the row 100 is 4 and the cell 4 is common to the second row and the first column of a three-variable K-map. In the example of Figure 5.4, a x is entered in that cell.

A	B	C	F	A \ BC	00	01	11	10
0	0	0	0	0	0	1	0	1
0	0	1	1					
0	1	0	1					
0	1	1	0					
1	0	0	x	1	x	0	1	x
1	0	1	0					
1	1	0	x					
1	1	1	1					

(a) Truth Table (b) K-map representing the truth table

Figure 5.4 Representation of truth table on a K-map.

5.4.3 Representation of Sum of Products Function on a K-map

Representation of Canonical Sum of Products Function on a K-map

A canonical sum of products function can be represented on a K-map by simply entering 1s in the cell of the K-map corresponding to each minterm present in the function. For example, the function

$$F = A'B'C + A'BC' + AB'C' + ABC$$

is represented on the K-map of Figure 5.5.

A \ BC	00	01	11	10
0		1		1
1	1		1	

Figure 5.5 Representation of the function

$F = A'B'C + A'BC' + AB'C' + ABC$ on a K-map.

Representation of Noncanonical Sum of Products Function on a K-map

In noncanonical sum of products function, some of the product terms have less than n literals, where n is the number of the variables in the function. These terms are represented on the K-map by more than one cell. For example, the function

$$F = \underline{A'C} + \underline{A'B} + \underline{AB'C} + \underline{BC}$$

is represented on the K-map of Figure 5.6. In Figure 5.6, to find the cells corresponding to $A'C$, we form the coincidence of A' (first row) and C (two middle columns) and obtain cells 001 and 011. 1s are entered in these cells. Note that when entering 1s in the cells, it is possible to find a 1 already placed there by a preceding term. The second term $A'B$ has 1s in cells 011 and 010, but cell 011 is common to the first term $A'C$ and only one 1 is marked in the cell 010 for the term $A'B$. The function has five minterms as indicated by the five cells marked with 1s. Note that the K-map is a useful tool for converting a noncanonical sum of products function into canonical sum of products function.

		B'		B	
		BC			
		00	01	11	10
A	$A' \rightarrow 0$		1 ✓	1 ✓	1 ✓
	$A \rightarrow 1$		1	1 ✓	
		$\uparrow C'$	C		$\uparrow C'$

Figure 5.6 Representation of the function $F = A'C + A'B + AB'C + BC$ on a K-map.

5.4.4 Representation of Product of Sums Function on a K-map

Representation of Canonical Product of Sums Function on a K-map

A canonical product of sums function can be represented on a K-map by entering 0s in the cells of K-map corresponding to each maxterm present in the function. For example, the function

$$F = (A + B + C)(A + B' + C')(A' + B + C')(A' + B' + C)$$

is represented on the K-map of Figure 5.7.

		BC			
		00	01	11	10
A	0	0		0	
	1		0		0

Figure 5.7 Representation of the function

$F = (A + B + C)(A + B' + C')(A' + B + C')(A' + B' + C)$ on a K-map.

Representation of Noncanonical Product of Sums Function on a K-map

In noncanonical product of sums function, some of the sum terms have less than n literals, where n is the number of the variables in the function. These terms are represented on the K-map by more than one cell. For example, the function

$$F = (B + C)(A' + B)(A + B' + C')$$

is represented in Figure 5.8. In Figure 5.8, to find the cells corresponding to $(B + C)$, we form the coincidence of BC (first column) and obtain cells 000 and 100. 0s are entered in these cells. The second term $(A' + B)$ has 0s in cells 100 and 101, but cell 100 is common to the first term $(B + C)$ and only one 0 is entered in cell 101 for the term $(A' + B)$. The function has four maxterms as indicated by the four cells marked with 0s. Also note that the K-map is a useful tool for converting noncanonical product of sums function into canonical product of sums function.

		BC		B		B'	
				00	01	11	10
A	$A \rightarrow 0$	0			0		
	$A' \rightarrow 1$	0	0				
		$\uparrow$	$\uparrow$			$\uparrow$	
		C	C'			C	

Figure 5.8 Representation of the function $F = (B + C)(A' + B)(A + B' + C')$ on a K-map.

5.4.5 Grouping of Adjacent 1s on a K-map

The property that *two horizontally or vertically adjacent cells differ in only one variable* provides a means of grouping adjacent 1s and simplifying the terms represented by the 1s of that group.

Group of One 1

A single 1 constitutes a group of one 1. A group of one 1 represents a minterm, that is, a product term containing n literals, where n is the number of variables in the function. A group of one 1 is shown in Figure 5.9(a).

Group of Two 1s

Any two adjacent 1s constitute a group of two 1s. In a group of two 1s, the two minterms can be combined to produce a single product term containing $n - 1$ literals. The two minterms differ in only one variable, that is, they are of the form

Ax and Ax' , where A is a product term independent of x . These two terms can be combined to produce the product term A as follows:

$$Ax + Ax' = A(x + x') = A \cdot 1 = A.$$

A group of two 1s is shown in Figure 5.9(a) and their combining is as follows:

$$A'B'CD' + A'BCD' = A'CD'(B' + B) = A'CD'$$

Group of Four 1s

Any four adjacent 1s, which form a square or a rectangle, constitute a group of four 1s. In a group of four 1s, the four minterms can be combined to produce a single product term containing $n - 2$ literals. The four minterms differ in two variables, that is, they are of the form $Ax'y'$, $Ax'y$, Axy' , and Axy , where A is a product term independent of x and y . These four minterms can be combined to produce the product term A as follows:

$$\begin{aligned} Ax'y' + Ax'y + Axy' + Axy &= Ax'(y' + y) + Ax(y' + y) = Ax' + Ax \\ &= A(x' + x) = A \end{aligned}$$

A group of four 1s is shown in Figure 5.9(a) and their combining is as follows:

$$\begin{aligned} ABC'D + ABCD + AB'C'D + AB'CD &= ABD(C' + C) + AB'D(C' + C) \\ &= ABD + AB'D \\ &= AD(B + B') \\ &= AD \end{aligned}$$

Group of Eight 1s

Any eight adjacent 1s, which form a rectangle, constitute a group of eight 1s. In a group of eight 1s, the eight minterms differ in three variables and they can be combined to produce a single product term containing $n - 3$ literals. A group of eight 1s is shown in Figure 5.9 (b) and their combining is as follows:

$$\begin{aligned} A'B'C'D + A'B'CD + A'BC'D + A'BCD + ABC'D + ABCD + AB'C'D + AB'CD \\ &= A'B'D(C' + C) + A'BD(C' + C) + ABD(C' + C) + AB'D(C' + C) \\ &= A'B'D + A'BD + ABD + AB'D \\ &= A'D(B' + B) + AD(B' + B) \\ &= A'D + AD \\ &= D(A' + A) \\ &= D \end{aligned}$$

In general, in an n -variable K-map any 2^m adjacent 1s, which form a square or a rectangle, constitute a group of 2^m 1s. The 2^m minterms of the group differ in m

variables and they can be combined to produce a single product term containing $n - m$ literals. A group of 2^n 1s combines 2^n minterms to produce a constant 1.

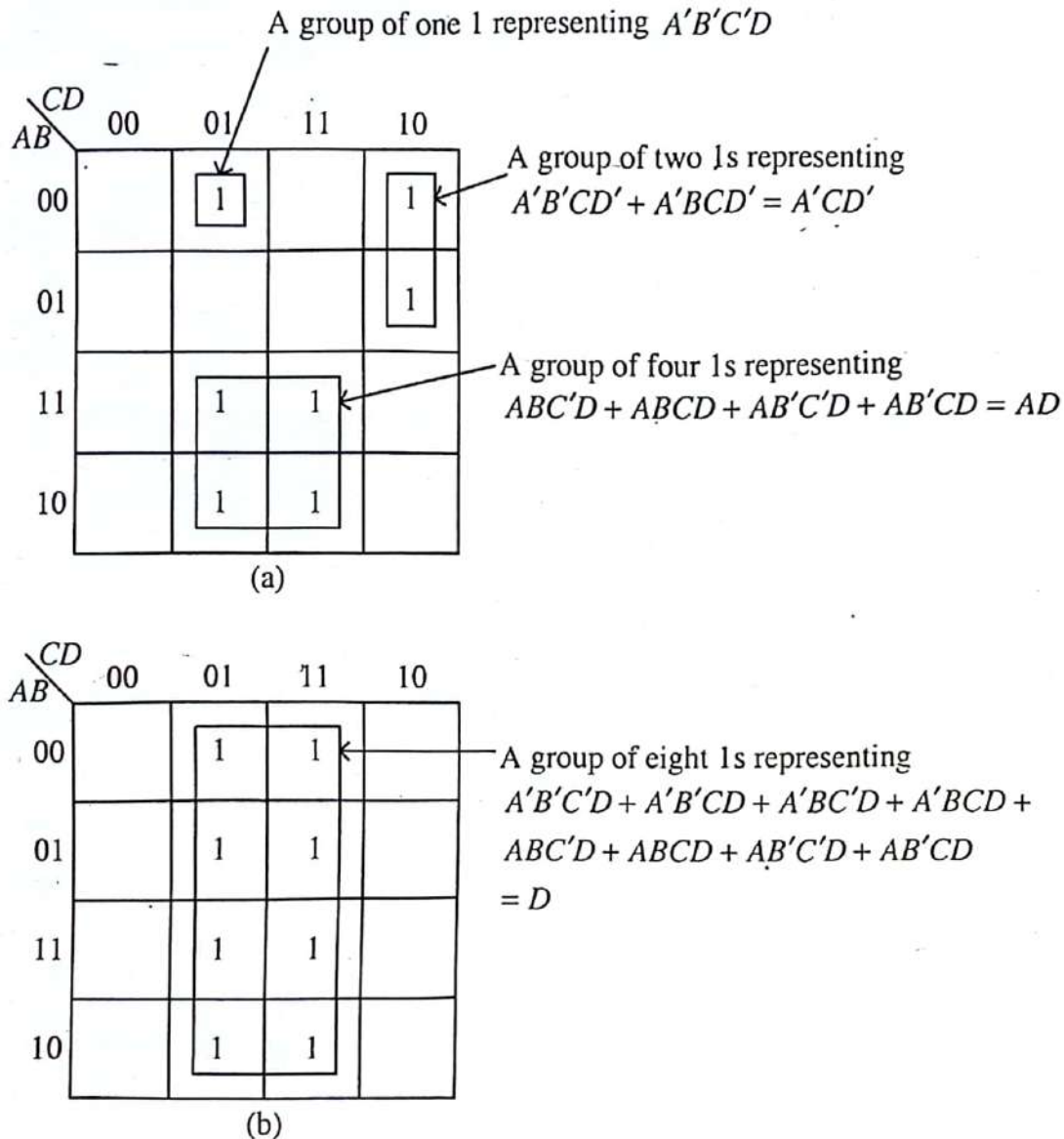


Figure 5.9 Grouping of adjacent 1s on a K-map for simplification.

Reading Product Term from a Group of 1s

The product term specified by a group of 1s can be read directly from the group. The column and row markings of the group are inspected for any variation in bit positions. The variable corresponding to a varying bit position does not appear in the product term. The literals corresponding to the non-varying bit positions are taken for constructing the product term. Note that a variable appears uncomplemented in the product term if its value is 1 and complemented if 0. For example, consider the group of four 1s in Figure 5.9(a). The column markings of this group are 01 and 11. Here the variable C varies and will not appear in the resulting product term. Similarly, the row markings of the group are 11 and 10 and the varying variable B will not appear in the product term. Thus, the resulting product term will be AD .

Don't Care Terms in a Group of 1s

The output for a don't care term is unspecified. A cell containing a don't care term can be assumed to contain a 1 and can be included in a group. Such inclusion will minimize the function though the operation of the corresponding digital circuit will not be affected, since the don't care term will never appear in the input of the circuit. Inclusion of a don't care term in a group is shown in Figure 5.10. It should be noted that a group of all don't care terms are not considered, that is, in a group there must be at least one 1. This is shown in Figure 5.10.

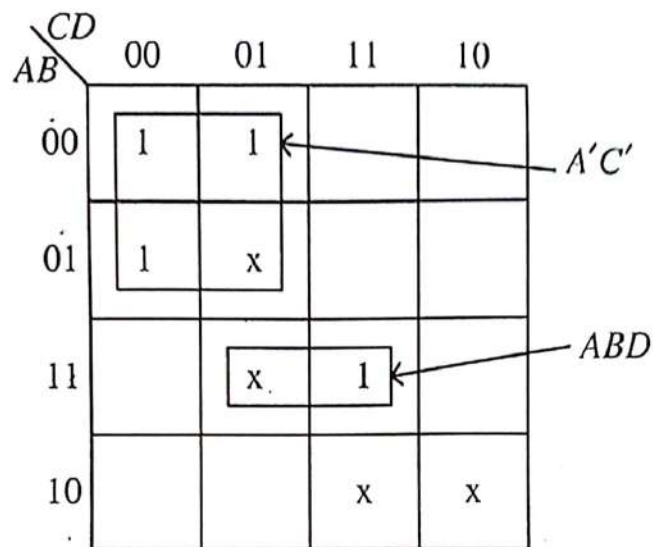


Figure 5.10 Don't care terms in a group of 1s.

Overlapping Groups

A 1 or x can be included in more than one groups for facilitating simplification of the function, since $A + A + \dots + A + A$. Overlapping groups are shown in Figure 5.11.

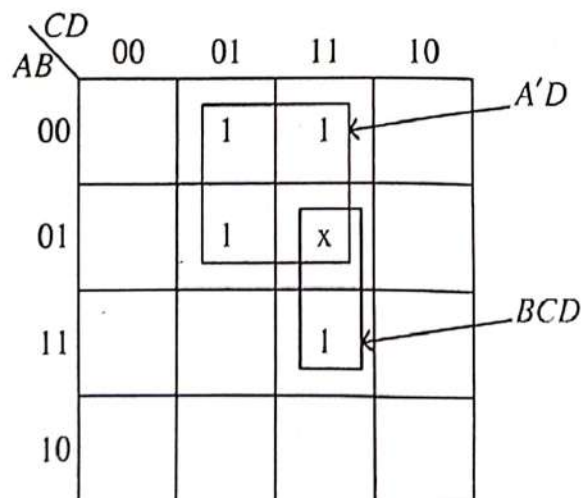


Figure 5.11 Overlapping groups on a K-map.

Prime Implicants on a K-map

A largest possible group of 1s (including any x) constitutes a prime implicant of the function. For example, consider the function represented by the K-map of Figure 5.12. The product term for group 1 is $A'BD$, for group 2 is ABD , and for group 3 is BD . The product terms $A'BD$ and ABD are not prime implicants of the function, since it is possible to delete a literal from them and still the resulting term is included in the function. But BD is a prime implicant of the function. Therefore, a group completely included within another group does not represent a prime implicant of the function. A group representing a prime implicant is an *essential prime implicant* if it covers at least one 1 not being covered by any other group.

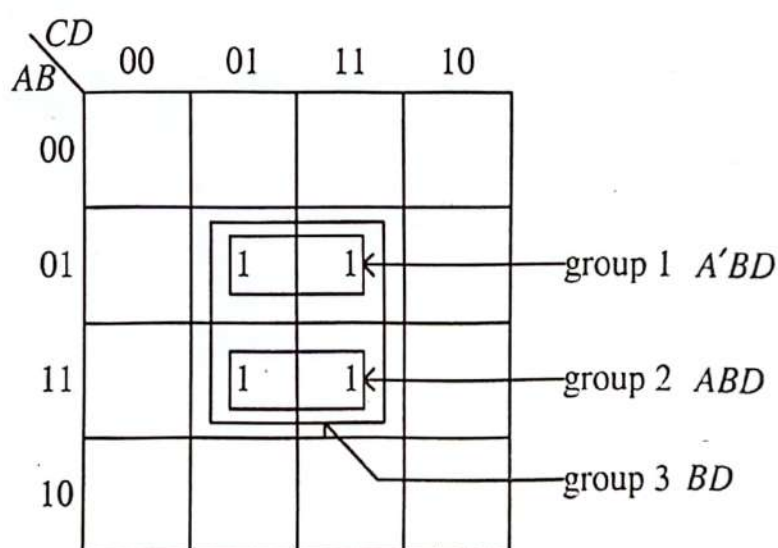


Figure 5.12 Prime Implicants on a K-map.

5.4.6 Grouping of Adjacent 0s on a K-map

In a K-map representing a product of sums function, the grouping of adjacent 0s is similar to the grouping of 1s. A group of one 0 represents a maxterm, that is, a sum term containing n literals, where n is the number of variables in the function. In a group of two 0s, the two maxterms differ in one variable and they are of the form $(A + x)$ and $(A + x')$, where A is a sum term independent of x . They can be combined to produce a single sum term A , containing $n - 1$ literals, as follows:

$$\begin{aligned}
 (A + x)(A + x') &= A \cdot A + A \cdot x' + A \cdot x + x \cdot x' \\
 &= A + A(x' + x) = A + A = A
 \end{aligned}$$

In general, in an n -variable K-map any 2^m adjacent 0s constitute a group of 2^m 0s; the 2^m maxterms of the group differ in m variables and they can be combined to produce a single sum term containing $n - m$ literals. A group of 2^n 0s combines 2^n maxterms to produce a constant 0.

The sum term specified by a group of 0s can be read directly from the K-map in a similar manner as reading a product term specified by a group of 1s. The variable corresponding to a varying bit position does not appear in the sum term. The literals corresponding to the non-varying bit positions are taken for constructing the sum term. Note that a variable appears uncomplemented in the sum term if its value is 0 and complemented if 1.

A cell containing a don't care term can be assumed to contain a 0 and can be included in a group of 0s in a similar as it is done in case of group of 1s. Overlapping groups are also considered in a similar manner as those are considered for groups of 1s. Groups of adjacent 0s are shown in Figure 5.13.

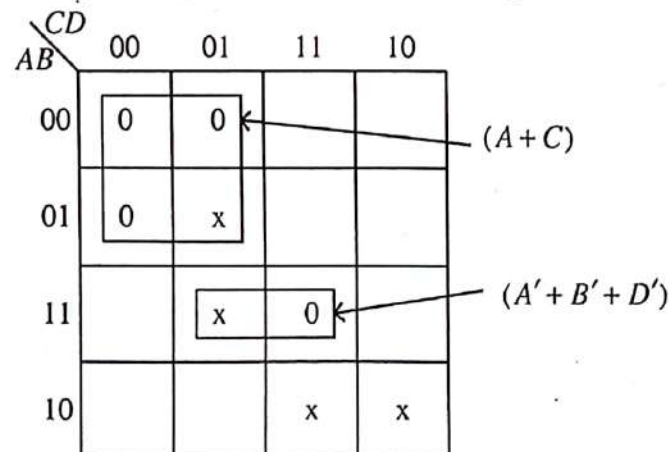


Figure 5.13 Grouping of adjacent 0s on a K-map for simplification.

5.4.7 Simplification of Sum of Products Function using K-map

Simplification Procedure

The simplification of sum of products function using K-map consists of the following steps:

1. Represent the function on a K-map and find all possible groups representing prime implicants of the function.
2. Find all essential prime implicants.
3. Determine the 1s not covered by essential prime implicants. Choose a minimum number of non-essential prime implicants, which cover the remaining 1s. The non-essential prime implicants covering the remaining 1s are referred to as secondary-essential prime implicants.
4. Determine the simplified sum of products function by combining the essential and the secondary-essential prime implicants.

The above procedure is illustrated by the following examples.

Example 1: $F(A, B, C, D) = \sum(0, 2, 5, 7, 8, 10, 15)$

1. The function is represented on the K-map of Figure 5.14. Groups 1, 2, and 3 represent all possible prime implicants of the function. The product term for group 1 is $B'D'$, for group 2 is $A'BD$, and for group 3 is BCD .

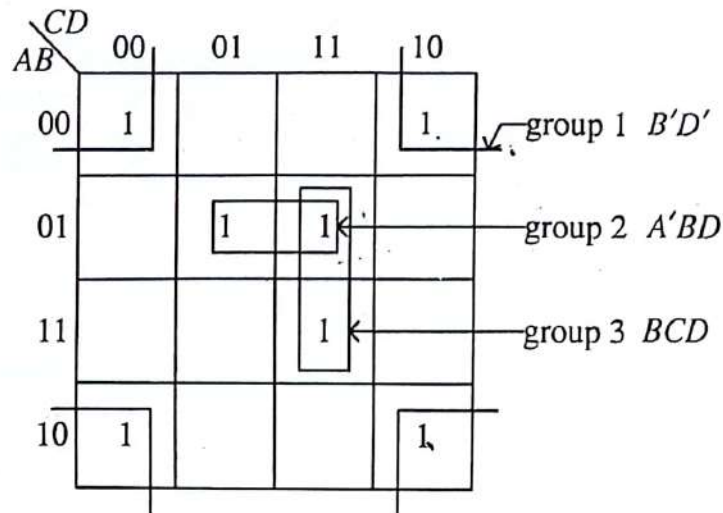


Figure 5.14 Simplification of sum of products function $F(A, B, C, D) = \sum(0, 2, 5, 7, 8, 10, 15)$ using K-map.

- Groups 1, 2, and 3 are all essential prime implicants.
- As all the prime implicants are essential prime implicants, all 1s are covered by them.
- The simplified sum of products function is

$$F(A, B, C, D) = B'D' + A'BD + BCD.$$

Example 2: $F(A, B, C, D) = \sum(1, 4, 5, 6, 7, 9, 10, 11, 13, 14)$

- The function is represented on the K-map of Figure 5.15. Groups 1, 2, 3, 4, 5, and 6 represent all possible prime implicants of the function. The product term for group 1 is $C'D$, for group 2 is $A'B$, for group 3 is BCD' , for group 4 is ACD' , for group 5 is $AB'C$, and for group 6 is $AB'D$.

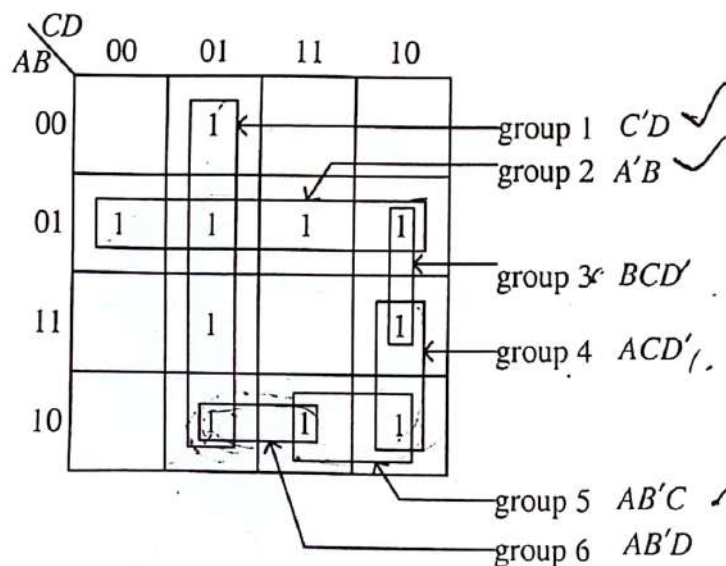


Figure 5.15 Simplification of sum of products function $F(A, B, C, D) = \sum(1, 4, 5, 6, 7, 9, 10, 11, 13, 14)$ using K-map.

2. Groups 1 and 2 are essential prime implicants.
3. Groups 3, 4, 5, and 6 are non-essential prime implicants. The 1s of the cells 1110, 1011, and 1010 are remained to be covered by a minimum subset of the non-essential prime implicants. The selection of the secondary-essential prime implicants can be done in three ways

- (i) Taking groups 3 and 5.
- (ii) Taking groups 4 and 6.
- (iii) Taking groups 4 and 5.

4. The simplified sum of products function is

$$F(A, B, C, D) = C'D + A'B + BCD' + AB'C$$

or

$$F(A, B, C, D) = C'D + A'B + ACD' + AB'D$$

or

$$F(A, B, C, D) = C'D + A'B + ACD' + AB'C$$

Example 3: $F(A, B, C, D) = \sum (0, 1, 3, 7, 8, 12) + \sum_{d.c.} (5, 10, 13, 14)$

1. The function is represented on the K-map of Figure 5.16. Groups 1, 2, 3, 4, and 5 represent all possible prime implicants of the function. The product term for group 1 is $A'D$, for group 2 is AD' , for group 3 is $A'B'C'$, for group 4 is $B'C'D'$, and for group 5 is ABC' .

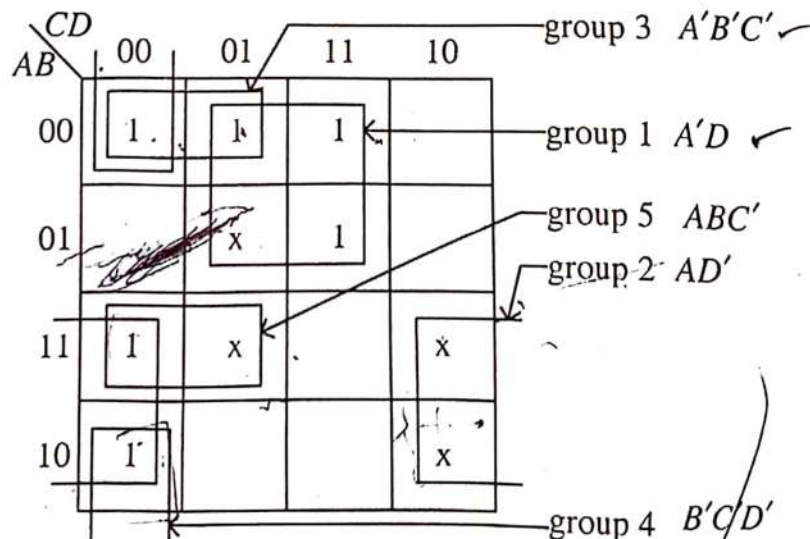


Figure 5.16 Simplification of sum of products function

$$F(A, B, C, D) = \sum (0, 1, 3, 7, 8, 12) + \sum_{d.c.} (5, 10, 13, 14) \text{ using K-map.}$$

2. Group 1 is essential prime implicant.
3. Groups 2, 3, 4, and 5 are non-essential prime implicants. The 1s of the cells 0000, 1100, and 1000 are left to be covered. The 1s of the cells 1100 and 1000 can be covered by choosing group 2. The 1 of the cell 0000 can be covered by choosing either group 3 or 4.

4. The simplified sum of products function is

$$F(A, B, C, D) = A'D + AD' + A'B'C'$$

or

$$F(A, B, C, D) = A'D + AD' + B'C'D'$$

✓ Straightforward Method of Simplification

The simplification procedure discussed above finds all possible simplified forms of the sum of products function. However, in practice, finding a single simplified sum of products function is sufficient. In this case, using intuition and experience, we try to directly identify groups representing essential and secondary-essential prime implicants only rather than identifying all possible groups representing all prime implicants. For this purpose, the following steps can be followed:

1. Represent the function on a K-map.
2. Identify the largest possible group, including don't care terms if possible, such that there is at least one 1 in the group.
3. Identify the next largest possible group, including don't care terms and already covered 1s if possible, such that there is at least one 1 not being covered by previous groups.
4. Repeat step 2 until all 1s are covered.
5. Discard a group if it contains no 1 not being covered by other group.
6. Determine the simplified sum of products function using the product terms of the selected groups.

The above procedure is discussed with the help of the following examples.

✓ **Example 1:** $F(A, B, C, D) = \sum(0, 1, 3, 7, 8, 12) + \sum_{d.c.}(5, 10, 13, 14)$

1. The function is represented on the K-map of Figure 5.17.

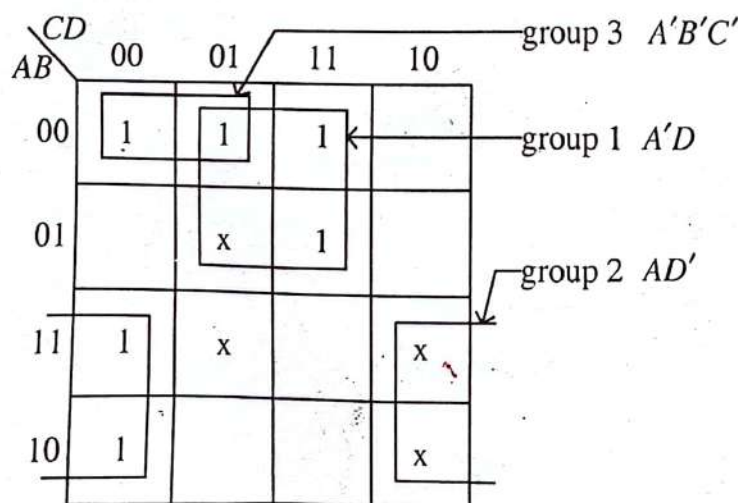


Figure 5.17 Simplification of sum of products function

$$F(A, B, C, D) = \sum(0, 1, 3, 7, 8, 12) + \sum_{d.c.}(5, 10, 13, 14) \text{ using K-map.}$$

2. First identify group 1, which contains three 1s and a don't care term. The product term for group 1 is $A'D$.
3. Then identify group 2, which contains two 1s and two don't care terms. The product term for group 2 is AD' .
4. Then identify group 3, which contains two 1s, among them one 1 is not covered by previous groups. The product term for group 3 is $A'B'C'$.
5. All groups contain at least one 1 not being covered by any other group.
6. The simplified sum of products function is

$$F(A, B, C, D) = A'D + AD' + A'B'C'.$$

Example 2: $F(A, B, C, D) = \sum(1, 5, 6, 7, 11, 12, 13, 15)$

1. The function is represented on the K-map of Figure 5.18.
2. First identify group 1, which contains four 1s. The product term for group 1 is BD .
3. Then identify group 2, which contains two 1s, among them one 1 is not covered by previous group. The product term for group 2 is $A'BC$.
4. Similarly identify groups 3, 4, and 5, which contains two 1s each, among them one 1 is not covered by previous groups. The product term for group 3 is ACD , for group 4 is ABC' , and for group 5 is $A'C'D$.
5. Group 1 contains four 1s, which are also contained in other groups. So, group 1 is discarded.
6. The simplified sum of products function is

$$F(A, B, C, D) = A'BC + ACD + ABC' + A'C'D.$$

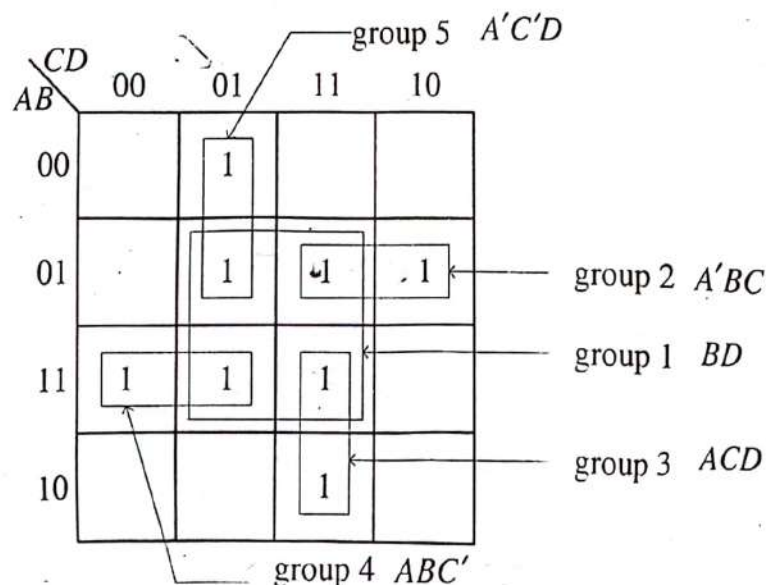


Figure 5.18 Simplification of sum of products function $F(A, B, C, D) = \sum(1, 5, 6, 7, 11, 12, 13, 15)$ using K-map.

5.4.8 Simplification of Product of Sums Function using K-map

Simplification Procedure

The simplification procedure for product of sums function is similar to that for sum of products function. The simplification of product of sums function using K-map consists of the following steps:

1. Represent the function on a K-map and find all possible groups.
2. Find all essential groups.
3. Determine the 0s not covered by essential groups. Choose a minimum number of non-essential groups, which cover the remaining 0s. The non-essential groups covering the remaining 0s are referred to as secondary-essential groups.
4. Determine the simplified product of sums function by combining the essential and the secondary-essential groups.

The above procedure is illustrated by the following example.

✓ Example: $F(A, B, C, D) = \prod (2, 4, 6, 9, 11, 15) \cdot \prod_{d.c.} (5, 10, 13, 14)$.

1. The function is represented on the K-map of Figure 5.19. Groups 1, 2, 3, 4, and 5 are the possible groups. The sum term for group 1 is $(C' + D)$, for group 2 is $(A' + D')$, for group 3 is $(A' + C')$, for group 4 is $(A + B' + D)$ and for group 5 is $(A + B' + C)$.
2. The groups 1 and 2 are essential groups.
3. Groups 3, 4, and 5 are non-essential groups. The cell 0100 containing 0 is left to be covered and can be done by choosing either group 4 or 5.
4. The simplified product of sums function is

$$F(A, B, C, D) = (C' + D)(A' + D')(A + B' + D)$$

or

$$F(A, B, C, D) = (C' + D)(A' + D')(A + B' + C)$$

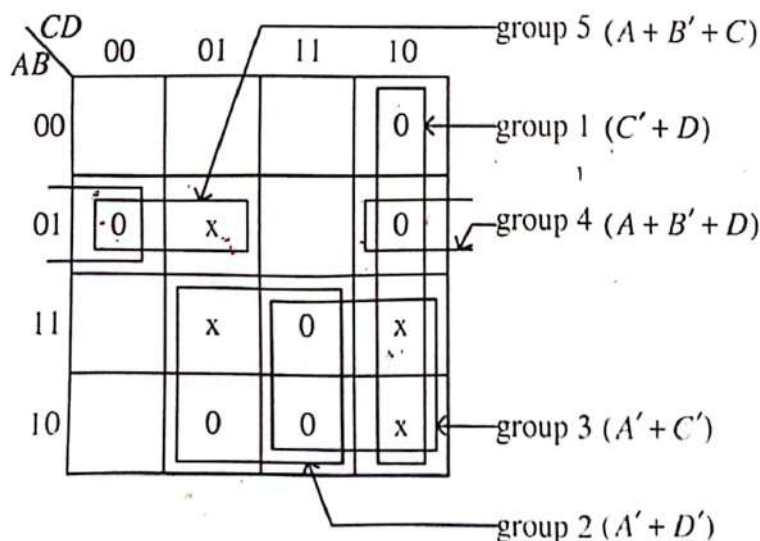


Figure 5.19 Simplification of

$$F(A, B, C, D) = \prod (2, 4, 6, 9, 11, 15) \cdot \prod_{d.c.} (5, 10, 13, 14) \text{ using K-map.}$$

✓ Straightforward Method of Simplification

The straightforward method of finding a single simplified product of sums function is also similar to that of sum of products function. For this purpose, the following steps can be followed:

1. Represent the function on a K-map.
2. Identify the largest possible group, including don't care terms if possible, such that there is at least one 0 in the group.
3. Identify the next largest possible group, including don't care terms and already covered 0s if possible, such that there is at least one 0 not being covered by previous groups.
4. Repeat step 2 until all 0s are covered.
5. Discard a group if it contains no 0 not being covered by other group.
6. Determine the simplified product of sums function using the sum terms of the selected groups.

The above procedure is discussed with the help of the following examples.

✓ **Example:** $F(A, B, C, D) = \prod (2, 4, 6, 9, 11, 15) \cdot \prod_{d.c.} (5, 10, 13, 14)$

1. The function is represented on the K-map of Figure 5.20.
2. First identify group 1, which contains two 0s and two don't care terms. The sum term for group 1 is $(C + D')$.
3. Then identify group 2, which contains three 0s and a don't care term. The sum term for group 2 is $(A' + D')$.
4. Then identify group 3, which contains two 0s, among them one 0 is not covered by previous groups. The sum term for group 3 is $(A + B' + D)$.
5. All groups contain at least one 0 not being covered by any other group.
6. The simplified sum of products function is

$$F(A, B, C, D) = (C + D')(A' + D')(A + B' + D).$$

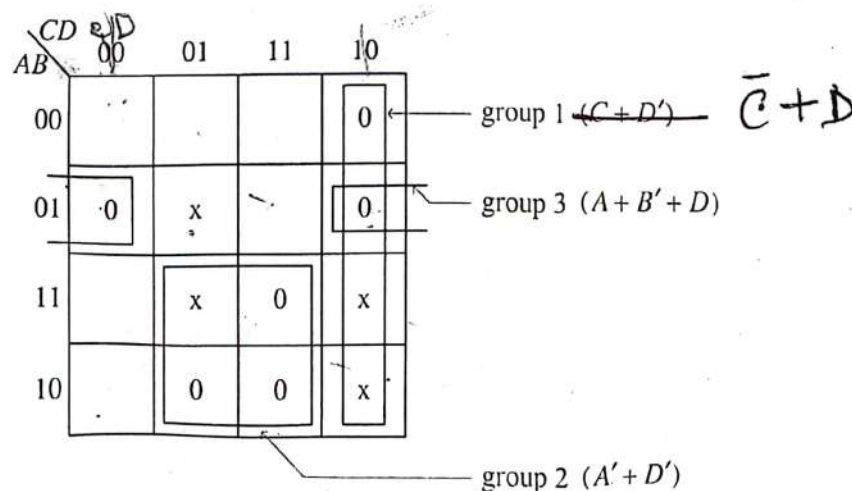


Figure 5.20 Simplification of

$F(A, B, C, D) = \prod (2, 4, 6, 9, 11, 15) \cdot \prod_{d.c.} (5, 10, 13, 14)$ using K-map.

5.4.7 Simplification of Sum of Products vs. Product of Sums Function

Simplification of Boolean function

$$F(A, B, C, D) = \sum(0, 1, 2, 3, 5, 7, 8, 9, 11, 14) = \prod(4, 6, 10, 12, 13, 15)$$

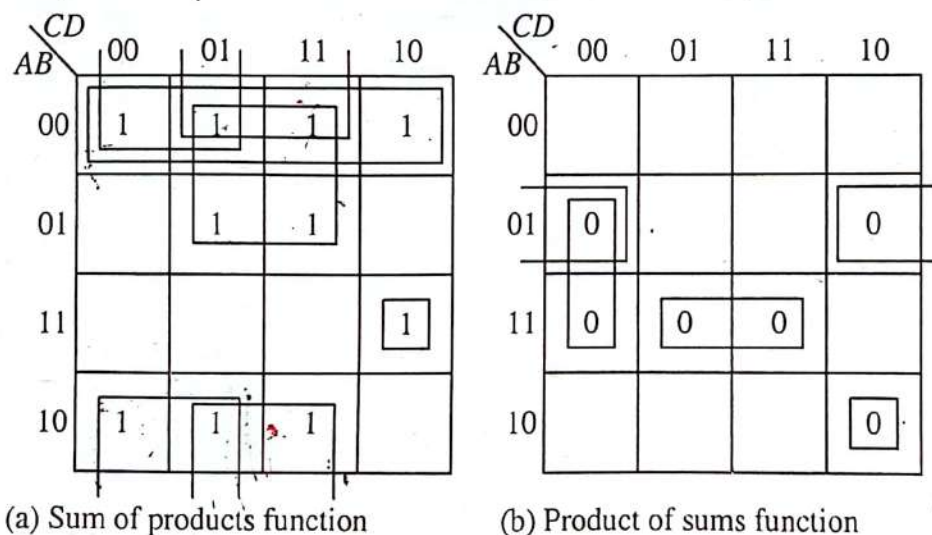
is shown in Figure 5.21. Simplification of sum of products function is shown in Figure 5.21(a), which yields

$$F(A, B, C, D) = A'B' + A'D + B'C' + B'D + ABCD'$$

Simplification of the product of sums function is shown in Figure 5.21(b), which yields

$$F(A, B, C, D) = (B' + C + D)(A' + B' + D')(A + B' + D)(A' + B + C' + D)$$

We observe that the numbers of terms are not same in the two simplifications. In fact, in general the two simplifications will not have the same number of terms and will require different quantities of gates. Therefore, one can obtain both simplifications and select the one that requires minimum gates.



$$F(A, B, C, D) = A'B' + A'D + B'C' + B'D + ABCD'$$

$$F(A, B, C, D) = (B' + C + D)(A' + B' + D')(A + B' + D)(A' + B + C' + D)$$

Figure 5.21 Simplification of

$$F(A, B, C, D) = \sum(0, 1, 2, 3, 5, 7, 8, 9, 11, 14) = \prod(4, 6, 10, 12, 13, 15)$$

5.5 Quine-McCluskey Method for Simplification of Boolean Function

5.5.1 The Quine-McCluskey Method

The K-map method is a very useful tool for the simplification of Boolean functions but suffers from some drawbacks. Firstly it depends upon human intuitiveness in forming groups of cells containing 1 or 0. Also the method cannot be applied for the functions having more than six variables and working with five and six variable functions is very cumbersome. The *Quine-McCluskey method*, also known as

tabulation method, is applicable to functions of any number of variables. It is a specific step-by-step procedure that is guaranteed to produce a simplified sum of products expression for a function. The procedure is also suitable for programming on a digital computer for systematic simplification of large Boolean functions. The method was first formulated by Quine and later improved by McCluskey.

The method consists of the following steps:

1. A set of all prime implicants of the function is obtained.
2. Essential prime implicants and a minimum number of secondary-essential prime implicants are selected so that they cover the function.

5.5.2 Determination of all Prime Implicants

The principle involved in the determination of all prime implicants is the repeated applications of the fact that

$$Ax + Ax' = A(x + x') = A$$

where A is a product term independent of x , on all adjacent pair of product terms so that they generate the set of all prime implicants.

Determination of all Prime Implicants using Binary Notations of Minterms

The procedure for determination of all prime implicants for the function is as follows:

1. Convert the Boolean function into sum of minterms form, if it is not.
2. Express each minterm by its binary representation. Arrange the minterms according to increasing index, where index is defined as the number of 1s in a minterm. Separate the sets of minterms of equal index by lines.
3. Compare each term of index n with each term of index $(n + 1)$. For each pair of terms that can combine, write the newly formed terms. Only those terms can be combined that differ only at one bit position. A dash is placed at that bit position in the newly formed term that has been eliminated as a result of combination. The terms that are combined are checked. After all pairs of terms with indices n and $(n + 1)$ have been considered, a line is drawn under the last term. The above procedure is repeated for all groups of terms. This completes one stage of elimination.
4. The next stage of elimination is done by using step 3. In this case two terms can be combined only when they have dashes in the same positions. Checks are placed as before. If same term is generated more than once, only one is listed.
5. Repeat step 4 until no new list can be formed.
6. All terms that remain unchecked are prime implicants.

The above procedure is explained for the Boolean function

$$F(A, B, C, D, E) = \sum(0, 2, 4, 6, 7, 8, 10, 11, 12, 13, 14, 16, 18, 19, 29, 30).$$

Step 1: The function is already in sum of minterms form.

Step 2: The result of step 2 is shown in the columns labeled "Index" and "Minterms" of Table 5.1.

Step 3: The result of step 3 is shown in the column labeled "Stage I" of Table 5.1.

Step 4 and 5: The result of step 4 and 5 is shown in the columns labeled "Stage II" and "Stage III" of Table 5.1.

Table 5.1 Determination of all prime implicants of function
 $F(A, B, C, D, E) = \sum(0, 2, 4, 6, 7, 8, 10, 11, 12, 13, 14, 16, 18, 19, 29, 30)$

Index	Minterms A B C D E	Stage I	Stage II	Stage III
0	0 00000✓	000-0✓	00--0✓	0---0 <i>a</i>
1	2 00010✓	00-00✓	0-0-0✓	
	4 00100✓	0-000✓	-00-0 <i>b</i>	
	8 01000✓	-0000✓	0--00✓	
	16 10000✓	00-10✓ (2,6)	0--10✓	
	5 00110✓	0-010✓ (2,6)	0-1-0✓	
2	10 01010✓	-0010✓ (2,6)	01--0✓	
	12 01100✓	001-0✓ (4,6)		
	18 10010✓	0-100✓ (4,6)		
	7 00111✓	010-0✓ (8,10)		
3	11 01011✓	01-00✓ (8,10)		
	13 01101✓	100-0✓ (16,18)		
	14 01110✓	0011- <i>h</i>		
	19 10011✓	0-110✓		
4	29 11101✓	0101- <i>g</i>		
	30 11110✓	01-10✓		
		0110- <i>f</i>		
		011-0✓		
		1001- <i>e</i>		
		-1101 <i>d</i>		
		-1110 <i>c</i>		

Step 6: All the unchecked terms are the prime implicants of the function. These are marked from *a* to *h* in Table 5.1. Note that the variable corresponding to a dash is missing from the product terms representing prime implicants. The prime implicants are listed below:

- a* 0---0 $A'E'$
- b* -00-0 $B'C'E'$
- c* -1110 $BCDE'$
- d* -1101 $BCD'E$
- e* 1001- $AB'C'D$
- f* 0110- $A'BCD'$
- g* 0101- $A'BC'D$
- h* 0011- $A'B'CD$



Determination of all Prime Implicants using Decimal Notations of Minterms

The above procedure shown in Table 5.1 consists of comparing terms and finding out terms that differ only at one bit position. This is quite cumbersome and error is likely to occur. The work is considerably reduced if decimal notation is used. Only decimal numbers of the minterms are written and the minterms are grouped according to the number of 1s in their binary representation as earlier. Now when the terms from the adjacent groups are combined, it should be remembered that the number of minterms of the next adjacent group that can combine must be larger by a power of 2, i.e., 1, 2, 4, 8, 16, 32, 64, ..., etc. Thus 2 can be combined with 3, 4, 6, 10, 18, 34, 66, ..., etc. It is quite simple to prove this fact because terms that can be combined must differ only at a single bit position, where the differing bit in the first number is 0 and that in the second number is 1. In binary numbers weight of one bit is equal to 2^i where $i = 0, 1, 2, 3, 4, 5, 6, \dots$. Therefore, in decimal number the second number will be larger by a power of 2.

Same example is repeated using decimal number in Table 5.2. The number by which two combining terms differ is placed in parenthesis. For example, when 8 and 10 combine, we write 8, 10 (2). Next combination is possible if the numbers in the parenthesis agree in two adjacent groups. The unchecked entries are prime implicants of the function and are marked from *a* to *h*.

Table 5.2 Determination of all prime implicants of function

$F(A, B, C, D, E) = \sum(0, 2, 4, 6, 7, 8, 10, 11, 12, 13, 14, 16, 18, 19, 29, 30)$ using decimal numbers.

Index	Minterms	Stage I	Stage II	Stage III
0	0 ✓	0, 2 (2) ✓	0, 2, 4, 6 (2, 4) ✓	0, 2, 4, 6, 8,
1	2 ✓	0, 4 (4) ✓	0, 2, 8, 10 (2, 8) ✓	10, 12, 14
	4 ✓	0, 8 (8) ✓	0, 2, 16, 18 (2, 16) <i>b</i>	(2, 4, 8) <i>a</i>
	8 ✓	0, 16 (16) ✓	0, 4, 8, 12 (4, 8) ✓	
	16 ✓	2, 6 (4) ✓	2, 6, 10, 14 (4, 8) ✓	
2	6 ✓	2, 10 (8) ✓	4, 6, 12, 14 (2, 8) ✓	
	10 ✓	2, 18 (16) ✓	8, 10, 12, 14 (2, 4) ✓	
	12 ✓	4, 6 (2) ✓		
	18 ✓	4, 12 (8) ✓		
3	7 ✓	8, 10 (2) ✓		
	11 ✓	8, 12 (4) ✓		
	13 ✓	16, 18 (2) ✓		
	14 ✓	6, 7 (1) <i>h</i>		
	19 ✓	6, 14 (8) ✓		
4	29 ✓	10, 11 (1) <i>g</i>		
	30 ✓	10, 14 (4) ✓		
		12, 13 (1) <i>f</i>		
		12, 14 (2) ✓		
		18, 19 (1) <i>e</i>		
		13, 29 (16) <i>d</i>		
		14, 30 (16) <i>c</i>		

1, 2, 4, 8, 16, 32, 64, ...

An easier method for writing the product terms of the prime implicants is to write down the binary equivalent of any minterm in the prime implicant and then put a dash in the positions whose binary weight corresponds to the numbers given in the parenthesis of the prime implicants. The prime implicants are listed below.

		2^4	2^3	2^2	2^1	2^0	
		16	8	4	2	1	
<i>a</i>	0, 2, 4, 6, 8, 10, 12, 14 (2, 4, 8)	0	0	0	0	0	$A'E'$
<i>b</i>	0, 2, 16, 18 (2, 16)	0	0	0	0	0	$B'C'E'$
<i>c</i>	14, 30 (16)	0	1	1	1	0	$BCDE'$
<i>d</i>	13, 29 (16)	0	1	1	0	1	$BCD'E$
<i>e</i>	18, 19 (1)	1	0	0	1	0	$AB'C'D$
<i>f</i>	12, 13 (1)	0	1	1	0	0	$A'BCD'$
<i>g</i>	10, 11 (1)	0	1	0	1	0	$A'BC'D$
<i>h</i>	6, 7 (1)	0	0	1	1	0	$A'B'CD$

Determination of all Prime Implicants of an Incompletely Specified Function

In case of incompletely specified Boolean function we start with the minterms and don't care terms together for determining all the prime implicants. This is illustrated using the following examples.

Example 1:

$$F(A, B, C, D, E) = \sum (1, 2, 3, 6, 8, 9, 14, 17, 24, 25, 26, 27, 30, 31) + \sum_{d.c.} (4, 5)$$

The determination of all prime implicants is shown in Table 5.3. The prime implicants are marked from *a* to *l* in Table 5.3 and listed below:

		16	8	4	2	1	
<i>a</i>	26, 27, 30, 31 (1, 4)	1	1	0	1	0	ABD
<i>b</i>	24, 25, 26, 27 (1, 2)	1	1	0	0	0	ABC'
<i>c</i>	8, 9, 24, 25 (1, 16)	0	1	0	0	0	$BC'D'$
<i>d</i>	1, 9, 17, 25 (8, 16)	0	0	0	0	1	$C'D'E'$
<i>e</i>	14, 30 (16)	0	1	1	1	0	$BCDE'$
<i>f</i>	6, 14 (8)	0	0	1	1	0	$A'CDE'$
<i>g</i>	4, 6 (2)	0	0	1	0	0	$A'B'CE'$
<i>h</i>	4, 5 (1)	0	0	1	0	0	$A'B'CD'$
<i>i</i>	2, 6 (4)	0	0	0	1	0	$A'B'DE'$
<i>j</i>	2, 3 (1)	0	0	0	1	0	$A'B'C'D'$
<i>k</i>	1, 5 (4)	0	0	0	0	1	$A'B'D'E$
<i>l</i>	1, 3 (2)	0	0	0	0	1	$A'B'C'E$

Table 5.3 Determination of all prime implicants of the function
 $F(A, B, C, D, E) = \sum (1, 2, 3, 6, 8, 9, 14, 17, 24, 25, 26, 27, 30, 31) + \sum_{d.c.} (4, 5)$

Index	Minterms	Stage I	Stage II
1	1 ✓	1, 3 (2) l	1, 9, 17, 25 (8, 16) d
	2 ✓	1, 5 (4) k	8, 9, 24, 25 (1, 16) c
	4 ✓	1, 9 (8) ✓	24, 25, 26, 27 (1, 2) b
	8 ✓	1, 17 (16) ✓	26, 27, 30, 31 (1, 4) a
2	3 ✓	2, 3 (1) j	
	5 ✓	2, 6 (4) i	
	6 ✓	4, 5 (1) h	
	9 ✓	4, 6 (2) g	
	17 ✓	8, 9 (1) ✓	
	24 ✓	8, 24 (16) ✓	
3	14 ✓	6, 14 (8) f	
	25 ✓	9, 25 (16) ✓	
	26 ✓	17, 25 (8) ✓	
4	27 ✓	24, 25 (1) ✓	
	30 ✓	24, 26 (2) ✓	
5	31 ✓	14, 30 (16) e	
		25, 27 (2) ✓	
		26, 27 (1) ✓	
		26, 30, (4) ✓	
		27, 31 (4) ✓	
		30, 31 (1) ✓	

Example 2: $F(A, B, C, D) = \sum (1, 3, 5, 7, 9, 11, 15) + \sum_{d.c.} (4, 6, 8, 14)$

The determination of all prime implicants is shown in Figure 5.4. The prime implicants are marked from a to f in Table 5.4 and listed below:

		8 4 2 1	
a	6, 7, 14, 15 (1, 8)	0 1 1 0	BC
b	3, 7, 11, 15 (4, 8)	0 0 1 1	CD
c	4, 5, 6, 7 (1, 2)	0 1 0 0	$A'B$
d	1, 3, 9, 11 (2, 8)	0 0 0 1	$B'D$
e	1, 3, 5, 7 (2, 4)	0 0 0 1	$A'D$
f	8, 9 (1)	1 0 0 0	$AB'C'$

Table 5.4 Determination of all prime implicants of the function
 $F(A, B, C, D) = \sum (1, 3, 5, 7, 9, 11, 15) + \sum_{d.c.} (4, 6, 8, 14)$

Index	Minterms	Stage I	Stage II
1	1 ✓	1, 3 (2) ✓	1, 3, 5, 7, (2, 4) <i>e</i>
	4 ✓	1, 5 (4) ✓	1, 3, 9, 11 (2, 8) <i>d</i>
	8 ✓	1, 9 (8) ✓	4, 5, 6, 7 (1, 2) <i>c</i>
	-----	4, 5 (1) ✓	-----
2	3 ✓	4, 6 (2) ✓	3, 7, 11, 15 (4, 8) <i>b</i>
	5 ✓	8, 9 (1) ✓ <i>f</i>	6, 7, 14, 15 (1, 8) <i>a</i>
	6 ✓	-----	
	9 ✓	3, 7 (4) ✓	
3	-----	3, 11 (8) ✓	
	7 ✓	5, 7 (2) ✓	
	11 ✓	6, 7 (1) ✓	
	14 ✓	6, 14 (8) ✓	
4	-----	9, 11 (2) ✓	
	15 ✓	-----	
		7, 15 (8) ✓	
		11, 15 (4) ✓	
		14, 15 (1) ✓	

5.5.3 Selection of Prime Implicants

The selection of prime implicants for covering the function consists of the following steps:

1. From the set of all prime implicants, a set of essential prime implicants is determined by making a prime implicant chart.
2. The minterms that are not covered by essential prime implicants are then considered and a minimum cover (secondary-essential prime implicants) is obtained from the remaining prime implicants.

The prime implicant chart is a table. The rows of the table correspond to the prime implicants arranged in order of increasing cost, where *cost* is the number of literals in the prime implicant. The columns correspond to minterms of the function that are to be covered. If the function has some don't care terms, they are not included in the prime implicant chart, since they need not to be covered, however, they must be considered for finding out the prime implicants. There is an extra column in the left and an extra row at the bottom. In each row excepting the last row "x" are placed in the columns corresponding to the minterms covered by the prime implicant in that row. The prime implicant chart for the example of Table 5.2 is shown in Table 5.5.

Table 5.5 Prime implicant chart for the example of Table 5.2.

		0	2	4	6	7	8	10	11	12	13	14	16	18	19	29	30
*	<i>a</i> 0,2,4,6,8,10,12,14 (2,4,8)	x	x	(x)	x		(x)	x		x		x					
*	<i>b</i> 0,2,16,18 (2,16)	x	x										(x)	x			
*	<i>c</i> 14,30 (16)											x					(x)
*	<i>d</i> 13,29 (16)										x					(x)	
*	<i>e</i> 18,19 (1)													x	(x)		
	<i>f</i> 12,13 (1)									x	x						
*	<i>g</i> 10,11 (1)							x	(x)								
*	<i>h</i> 6,7 (1)				x	(x)											
		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

After completing the table for all prime implicants of the function, a search is made for the column having a single "x" and they are encircled. In this example we see that there is only one "x" in the columns corresponding to minterms 4, 7, 8, 11, 16, 19, 29, and 30. The prime implicants that cover these minterms are essential prime implicants, since these prime implicants are the only one covering the corresponding minterms. An asterisk is placed in the left most column on the row corresponding to essential prime implicants. We also place checks in the bottom row in columns of the minterms that are covered by the essential prime implicants. The essential prime implicants for this example are *a*, *b*, *c*, *d*, *e*, *g*, and *h*. In this example, we see that all the minterms are covered by the essential prime implicants. The prime implicant *f* is redundant because minterms 12 and 13 are already covered by essential prime implicants. The simplified sum of product expression for the function is, thus, given by

$$F(A, B, C, D, E) = a + b + c + d + e + g + h$$

$$= A'E' + B'C'E' + BCDE' + BCD'E + AB'C'D + A'BC'D + A'B'CD$$

In the above example we see that all the minterms are covered by the essential prime implicants, but it may not be so for all the cases. In general, we are left with some unchecked columns and a set of non-essential prime implicants. The next step is to select the minimum set of secondary-essential prime implicants out of the remaining non-essential prime implicants so as to cover the unchecked minterms. The procedure for this is as follows:

1. Make a reduced prime implicant chart containing non-essential prime implicants and the unchecked minterms.
2. In the reduced prime implicant chart, if two prime implicants cover the same set of minterms, the two corresponding rows are called equivalent rows. We can cancel one prime implicant which is having more cost.
3. We can also cancel the non-dominate row B, where a row A is said to dominate over row B, if row A has "x" in all the columns in which row B has "x" and at the same time A also has "x" in at least one column in which B does not have "x".

4. After canceling the equivalent rows and non-dominate rows, we carry out the same procedure, as we do in selecting essential prime implicants for covering the rest of the minterms.

The above procedure is explained using the prime implicants of Table 5.3. The prime implicant chart is shown in Table 5.6. In this table we make columns for the minterms only.

Table 5.6 Prime implicant chart for the example of Table 5.3.

		1	2	3	6	8	9	14	17	24	25	26	27	30	31
*	<i>a</i> 26,27,30,31 (1,4)											x	x	x	(x)
	<i>b</i> 24,25,26,27 (1,2)									x	x	x	x		
*	<i>c</i> 8,9,24,25 (1,16)					(x)	x			x	x				
*	<i>d</i> 1,9,17,25 (8,16)	x					x		(x)		x				
	<i>e</i> 14,30 (16)							x						x	
	<i>f</i> 6,14 (8)					x		x							
	<i>g</i> 4,6 (2)				x										
	<i>h</i> 4,5 (1)														
	<i>i</i> 2,6 (4)		x		x										
	<i>j</i> 2,3 (1)		x	x											
	<i>k</i> 1,5 (4)	x													
	<i>l</i> 1,3 (2)	x		x											
		✓				✓	✓		✓	✓	✓	✓	✓	✓	✓

It can be seen from the prime implicant chart of Table 5.6 that *a*, *c*, and *d* are essential prime implicants. The reduced prime implicant chart is now made for the remaining minterms and prime implicants in Table 5.7.

Table 5.7 Reduced prime implicant chart for Table 5.6.

		2	3	6	14
	<i>b</i> 24,25,26,27 (1,2)				
	<i>e</i> 14,30 (16)				x
*	<i>f</i> 6,14 (8)			x	(x)
	<i>g</i> 4,6 (2)			x	
	<i>h</i> 4,5 (1)				
	<i>i</i> 2,6 (4)	x		x	
*	<i>j</i> 2,3 (1)	x	(x)		
	<i>k</i> 1,5 (4)				
	<i>l</i> 1,3 (2)				x
		✓	✓	✓	✓

It can be seen from Table 5.7 that prime implicant *e* is dominated by *f*, *g* is dominated by both *f* and *i*, and *l* is dominated by *j*. Therefore, prime implicants *e*, *g*, and *l* are canceled out. Now we look for the columns in which there is a single "x"

to find out secondary-essential prime implicants. It can be seen that f and j are secondary-essential prime implicants and they cover rest of the minterms. The simplified sum of product expression for the function is, thus, given by

$$F(A, B, C, D) = a + c + d + f + j$$

$$= ABD + BC'D' + C'D'E + A'CDE' + A'B'C'D$$

5.5.4 Cyclic Prime Implicant Chart

In some prime implicant charts or sometimes after removing equivalent and non-dominant rows in the reduced prime implicant charts, we may be left with more than one check in each column so that it becomes difficult to select the proper prime implicants. Such prime implicant charts are called *cyclic prime implicant charts* and can be solved by using **Petrick's algebraic procedure**. The procedure is illustrated using the prime implicants for the example of Table 5.4. The prime implicant chart is shown in Table 5.8, which is a cyclic prime implicant chart.

Table 5.8 Cyclic prime implicant chart for example of Table 5.4:

		1.	3.	5	7	9	11	15
2	a 6, 7, 14, 15 (1, 8) ✓				x			x
2	b 3, 7, 11, 15 (4, 8) ✓		x		x		x	x
2	c 4, 5, 6, 7 (1, 2)			x	x			
2	d 1, 3, 9, 11 (2, 8) ✓	x	x			x	x	
2	e 1, 3, 5, 7 (2, 4) ✓	x	x	x	x			
3	f 8, 9 (1)					x		

From the column for minterm 1, it can be seen that for covering the minterm, either d or e should be selected. If all the prime implicants are taken as Boolean variables, then we must have $(d + e) = 1$. Similarly for covering minterm 3, $(b + d + e) = 1$. For covering both the minterms 1 and 3, $(d + e)(b + d + e) = 1$. In this way we get the complete expression as

$$(d + e)(b + d + e)(c + e)(a + b + c + e)(d + f)(b + d)(a + b) = 1$$

or,

$$(bd + d + de + be + \underline{de} + e)(\underline{ac} + \underline{bc} + c + \underline{ce} + ae + be + \underline{ce} + e)$$

$$(bd + d + bf + df)(a + b) = 1$$

$$\text{or, } (d + e)(c + e)(d + bf + df)(a + b) = 1$$

$$\text{or, } (cd + de + ce + e)(ad + bd + abf + bf + adf + bdf) = 1$$

$$\text{or, } (cd + e)(ad + bd + bf) = 1$$

$$\text{or, } acd + bcd + bcdf + ade + bde + bfe = 1$$

$$\text{or, } \underline{acd + bcd + ade + bde} + bfe = 1$$

$$= a + c + d$$

Therefore, at least one of the five products in the above equation should be 1. Since all products contain three prime implicants, the final selection will be based on the cost. The costs of prime implicants a to e is all 2 and the cost of prime implicant f is 3. Therefore, the products acd , bcd , ade , and bde all cost 6 and the product bfe costs 7. As the first four products cost minimum, any of them can be taken for the minimum cover. If we take prime implicants a , c , and d for the minimum cover, the simplified function is

$$F(A, B, C, D) = a + c + d = BC + A'B + B'D$$

Thus, the Petrick's algebraic method gives all possible combinations of prime implicants for covering all the minterms and a combination having minimum number of prime implicants and a minimum cost is selected out of them. For equal cost any combination can be selected.

✓ 5.6 Iterated Consensus Method for Simplification of Boolean Function

5.6.1 The Consensus Operation

The consensus for two product terms T_1 and T_2 (denoted $T_1 \uparrow T_2$) is defined as

- (i) If there exists a literal x and product terms T_3 and T_4 , which are independent of x , such that $T_1 = x \cdot T_3$ and $T_2 = x' \cdot T_4$, then $T_1 \uparrow T_2 = T_3 T_4$.
- (ii) If the above condition is not satisfied, that is no variable appears complemented in one term and uncomplemented in the other, then $T_1 \uparrow T_2 = 0$.

For example, $x'_2 x_4 \uparrow x_1 x'_2 x'_4 = x'_2 \cdot x_1 x'_4 = x_1 x'_2$.

Properties of the Consensus Operation

The consensus operation is commutative but not associative, that is $T_1 \uparrow T_2 = T_2 \uparrow T_1$ but $(T_1 \uparrow T_2) \uparrow T_3 \neq T_1 \uparrow (T_2 \uparrow T_3)$. For example, if $T_1 = x'_2 x_4$, $T_2 = x_1 x'_2 x'_4$, and $T_3 = x_2 x_3$ then

$$\begin{aligned} T_1 \uparrow T_2 &= x'_2 x_4 \uparrow x_1 x'_2 x'_4 = x'_2 \cdot x_1 x'_4 = x_1 x'_2 \\ T_2 \uparrow T_1 &= x_1 x'_2 x'_4 \uparrow x'_2 x_4 = x_1 x'_2 \cdot x'_4 = x_1 x'_2 \end{aligned}$$

but

$$\begin{aligned} (T_1 \uparrow T_2) \uparrow T_3 &= (x'_2 x_4 \uparrow x_1 x'_2 x'_4) \uparrow x_2 x_3 = x_1 x'_2 \uparrow x_2 x_3 = x_1 x_3 \\ T_1 \uparrow (T_2 \uparrow T_3) &= x'_2 x_4 \uparrow (x_1 x'_2 x'_4 \uparrow x_2 x_3) = x'_2 x_4 \uparrow x_1 x_3 x'_4 = x_1 x'_2 x_3 \end{aligned}$$

Another important property of the consensus operation is that $T_1 \uparrow T_2 \subseteq T_1 + T_2$, which means that the term produced by the consensus of T_1 and T_2 is included in $T_1 + T_2$. For example, if $T_1 = x'_2 x_4$ and $T_2 = x_1 x'_2 x'_4$, then

$$T_1 \uparrow T_2 = x_1 x'_2 \subseteq x'_2 x_4 + x_1 x'_2 x'_4$$

5.6.2 Simplification of Boolean Function

The iterated consensus method of simplification of Boolean function depends on the following two steps:

1. Determine all prime implicants of the function by the repeated application of the consensus operation.
2. Determine a minimum cover of the function using the methods of sections 5.5.3 and 5.5.4.

Determination of All Prime Implicants

Unlike Quine-McCluskey method, the iterated consensus method does not require that the function be represented as sum of minterms. The simplification can be done using any sum of products form. The prime implicants of the function is determined using the following steps:

Step 1: Make a list of product terms included in the function.

Step 2: For each pair of terms, T_i and T_j (including the new terms added to the list) find out $T_i \uparrow T_j$ and add it to the list if $T_i \uparrow T_j \neq 0$.

Each time a term is added to the list, check out the terms which are equal to or included in the new term. Do not use any checked term for finding new consensus product. Care must be taken to insure that each unchecked term has been compared with all other unchecked terms in the list. For this purpose, starting from the second term, each unchecked term in the list is compared with all the unchecked terms above it. The process ends when all possible consensus operations have been performed.

Step 3: The unchecked terms in the list are the prime implicants of the function.

The procedure is illustrated for the function $f(x_1, x_2, x_3, x_4) = x'_2 x_4 + x_1 x'_2 x'_4 + x'_1 x_2 x_4 + x'_1 x'_2 x'_3$.

Step 1: The product terms of the function are listed above the horizontal line in Table 5.9.

Step 2: The outcome of step 2 is shown below the horizontal line in Table 5.9. Consensus operations of possible pairs are shown and necessary explanations are given in the right of each operation.

Table 5.9 Determination of prime implicants of the function

$f(x_1, x_2, x_3, x_4) = x_2'x_4 + x_1x_2'x_4' + x_1'x_2x_4 + x_1'x_2'x_3'$ using iterated consensus method.

Step 1: $T_1 : x_2'x_4 \quad \checkmark$
 $T_2 : x_1x_2'x_4' \quad \checkmark$
 $T_3 : x_1'x_2x_4 \quad \checkmark$
 $T_4 : x_1'x_2'x_3' \quad \checkmark$

Step 2: $T_5 : x_1x_2'$ $T_2 \uparrow T_1 = x_1x_2' = T_5 \supseteq T_2$ (T_5 is added to the list and T_2 is checked out)

$T_6 : x_1'x_4$ $T_3 \uparrow T_1 = x_1'x_4 = T_6 \supseteq T_3$ (T_6 is added to the list and T_3 is checked out)

$T_7 : x_2'x_3'$ $T_4 \uparrow T_1 = 0$
 $T_5 \uparrow T_1 = 0$
 $T_5 \uparrow T_4 = x_2'x_3' = T_7 \supseteq T_4$ (T_7 is added to the list and T_4 is checked out)

$T_8 : x_2'x_4$ $T_6 \uparrow T_1 = 0$
 $T_6 \uparrow T_5 = x_2'x_4 = T_8 = T_1$ (T_8 is added to the list and T_1 is checked out)

$T_7 \uparrow T_5 = 0$
 $T_7 \uparrow T_6 = 0$
 $T_8 \uparrow T_5 = 0$
 $T_8 \uparrow T_6 = 0$
 $T_8 \uparrow T_7 = 0$

Step 3: The prime implicants of the function are x_1x_2' , $x_1'x_4$, $x_2'x_3'$, and $x_2'x_4$.

Selection of prime Implicants

The selection of prime implicants is done by the methods of sections 5.5.3 and 5.5.4. However, for constructing the prime implicant chart, if the function is not already in the sum of minterms form, it is expanded into sum of minterms form. This can easily be done using K-map if the function is of 4 or less variables. If the function is of more than 4 variables, algebraic method can be used. The expansion of the function $f(x_1, x_2, x_3, x_4) = x_2'x_4 + x_1x_2'x_4' + x_1'x_2x_4 + x_1'x_2'x_3'$ using K-map is shown in Figure 5.21, which gives

$$f(x_1, x_2, x_3, x_4) = x_2'x_4 + x_1x_2'x_4' + x_1'x_2x_4 + x_1'x_2'x_3'$$

$x_1x_2 \backslash x_3x_4$	00	01	11	10
00	1	1	1	
01		1	1	
11				
10	1	1	1	1

$$f(x_1, x_2, x_3, x_4) = x_2'x_4 + x_1x_2'x_4' + x_1'x_2x_4 + x_1'x_2'x_3'$$

$$= \sum(0,1,3,5,7,8,9,10,11)$$

Figure 5.21 Expansion of $f(x_1, x_2, x_3, x_4) = x_2'x_4 + x_1x_2'x_4' + x_1'x_2x_4 + x_1'x_2'x_3'$ into sum of minterms form.

In the prime implicant chart, the prime implicants and the minterms are represented using their binary notations, for example, the minterm 0 is represented as 0000 and the prime implicant x_1x_2' is represented as 10—. A prime implicant covers a minterm if the non-missing literals of the prime implicant are exactly the same as those in the minterm. For example, the prime implicant 10— covers the minterms 1000, 1001, 1010, and 1011. The prime implicant chart for the example of Table 5.9 is shown in Table 5.10. In Table 5.10, the essential prime implicants are x_1x_2' , $x_1'x_4$, and $x_2'x_3'$, which covers all the minterms. Therefore, the simplified function is

$$f(x_1, x_2, x_3, x_4) = x_1x_2' + x_1'x_4 + x_2'x_3'$$

Table 5.10 Prime implicant chart for the example of Table 5.9

		0	1	3	5	7	8	9	10	11
		0000	0001	0011	0101	0111	1000	1001	1010	1011
*	$x_1x_2' (10--)$						x	x	(x)	x
*	$x_1'x_4 (0--1)$		x	x	(x)	(x)				
*	$x_2'x_3' (-00-)$	(x)	x				x	x		
	$x_2'x_4 (-0-1)$		x	x				x		x
		✓	✓	✓	✓	✓	✓	✓	✓	✓

The advantage of the iterated consensus method over Quine-McCuskey method is that usually fewer comparisons are needed to be made. Moreover, it is not necessary to expand the function to canonical sum of products form and can be started with any partially simplified function.

Exercises**5.1** Simplify algebraically the following functions.

(a) $F = (A' + B)(A + B + D)D'$

(b) $F = A'BC + AB'C + ABC' + ABC$

(c) $F = A'BCD + AB'C'D' + AB'C'D + AB'CD' + AB'CD + ABC'D' + ABC'D + ABCD' + ABCD$

(d) $F = ABC + A'C$

(e) $F = (A + B)(A' + B')$

(f) $F = (ABC)'(A + B + C)'$

(g) $F = (B + C')(B' + C) + (A' + B + C')'$

(h) $F = (C + D)' + A'CD' + AB'C' + A'B'CD + ACD'$

5.2 (a) Show that a group of 2^n 1s on an n -variable K-map produces a constant 1.(b) Show that a group of 2^n 0s on an n -variable K-map produces a constant 0.**5.3** Simplify the following sum of products functions using K-map method.

(a) $F(A, B, C, D) = \sum (0, 1, 2, 3, 7, 8, 9, 10, 11, 12, 13)$

(b) $F(A, B, C, D) = \sum (0, 2, 4, 6, 8) + \sum_{d.c.} (10, 11, 12, 13, 14, 15)$

(c) $F(A, B, C, D) = \sum (0, 2, 4, 9, 12, 15) + \sum_{d.c.} (1, 5, 7, 10)$

(d) $F(A, B, C, D) = \sum (0, 3, 4, 5, 6, 7, 8, 13, 14)$

(e) $F(A, B, C, D) = \sum (4, 6, 7, 9, 10, 11, 12, 14, 15)$

(f) $F(A, B, C, D) = \sum (0, 1, 2, 3, 4, 6, 7, 8, 9, 11, 15)$

(g) $F(A, B, C, D) = \sum (1, 3, 4, 5, 7, 8, 9, 11, 14, 15)$

(h) $F(A, B, C, D) = \sum (1, 2, 3, 4, 5, 6, 13, 14, 15)$

(i) $F(A, B, C, D) = ABC'D + A'BCD + A'B'C' + A'B'D' +$

$AC' + AB'C + B'$

(j) $F(A, B, C, D) = \sum (1, 3, 7, 11, 15) + \sum_{d.c.} (0, 2, 5)$

(k) $f(x, y, z) = x'yz + x'yz' + xy'z' + xy'z$

(l) $F(A, B, C) = A'C + A'B + AB'C + BC$

(m) $F(A, B, C, D) = A'B'C' + B'CD' + A'BCD' + AB'C'$

(n) $F(A, B, C, D) = \sum (3, 4, 5, 7, 9, 13, 14, 15)$

(o) $F(A, B, C, D) = \sum (0, 1, 2, 3, 5, 7, 8, 9, 11, 14)$

5.4 Simplify the following product of sums functions using K-map method.

(a) $F(A, B, C, D) = \prod (1, 4, 5, 6, 11, 12, 13, 14, 15)$

$$(b) F(A, B, C, D) = \prod (4, 5, 6, 7, 8, 12) \cdot \prod_{d.c.} (1, 2, 3, 9, 11, 14)$$

$$(c) F(A, B, C, D) = (A + B + C' + D')(A' + C + D')(A' + B + C' + D')(B' + C)(B' + C')(A + B')(B' + D')$$

5.5 Simplify the following functions using Quine-McCluskey method.

$$(a) F(A, B, C, D, E) = \sum (1, 3, 6, 10, 11, 12, 14, 15, 17, 19, 20, 22, 24, 29, 30)$$

$$(b) F(A, B, C, D, E) = \sum (1, 3, 6, 7, 8, 12, 14, 16, 18, 22, 23, 24, 28, 30, 31)$$

$$(c) F(A, B, C, D, E) = \sum (0, 1, 4, 5, 6, 7, 9, 11, 15) + \sum_{d.c.} (10, 14)$$

$$(d) F(A, B, C, D, E) = \sum (4, 8, 10, 16, 21, 27, 28) + \sum_{d.c.} (1, 5, 23, 25, 30, 31)$$

$$(e) F(A, B, C, D, E) = \sum (1, 5, 6, 12, 13, 14) + \sum_{d.c.} (2, 4)$$

$$(f) F(A, B, C, D) = \sum (1, 2, 3, 5, 13) + \sum_{d.c.} (6, 7, 8, 9, 11, 15)$$

$$(g) F(A, B, C, D, E) = \sum (0, 1, 2, 3, 4, 6, 9, 10, 15, 16, 17, 18, 20, 23, 25, 26, 31)$$

$$(h) F(A, B, C, D, E) = \sum (1, 5, 6, 7, 9, 13, 14, 15, 17, 18, 19, 21, 22, 23, 25, 29, 30)$$

$$(i) F(A, B, C, D, E) = \sum (8, 12, 13, 18, 19, 21, 22, 24, 25, 28, 31) + \sum_{d.c.} (1, 2, 3, 6, 7, 11)$$

5.6 Simplify the following functions using iterated consensus method.

$$(a) f(x_1, x_2, x_3, x_4) = x_1'x_2'x_3' + x_1'x_3x_4 + x_1'x_2x_4 + x_1x_2'$$

$$(b) f(x_1, x_2, x_3, x_4) = x_1'x_2x_3' + x_1x_2x_4 + x_2x_3x_4 + x_1x_2'x_3 + x_1'x_2'x_3x_4'$$

$$(c) F(A, B, C, D) = A'B'C' + BC'D' + A'CD + ACD + ACD'$$

$$(d) F(A, B, C, D) = A'C'D' + BC'D + B'C'D + ABC' + A'BD' + BCD'$$

$$(e) F(A, B, C, D, E) = \sum (0, 1, 4, 5, 6, 7, 9, 11, 15) + \sum_{d.c.} (10, 14)$$

$$(f) F(A, B, C, D) = \sum (0, 3, 4, 5, 6, 7, 8, 13, 14)$$

Further Reading

1. E. J. McCluskey, *Logic Design Principles*, Prentice-Hall, Inc., 1986.
2. Zvi Kohavi, *Switching and Finite Automata Theory*, 2nd ed., McGraw-Hill Higher Education, 1990.
3. Richard S. Sandige, *Modern Digital Design*, McGraw-Hill Publishing Company, 1990.
4. M. Morris Mano, *Digital Logic and Computer Design*, Prentice-Hall, Inc., 2000.
5. D. C. Green, *Digital Electronics*, 5th ed., Pearson Education, 2001.
6. V. K. Jain, *Switching Theory and Digital Electronics*, Khanna Publishers, India, 2002.